

B2 Task

Memory-Optimized Custom Container in C

Objective:

Design and implement a custom container in C that mimics the functionality of a dynamic array but with strict memory management optimizations. The container should demonstrate efficient use of memory, handling of edge cases, and optimized reallocation strategies.

Requirements:

1. Custom Dynamic Array Implementation:

- Implement a dynamic array (e.g., `CustomArray`) with the following functionalities:
 - Initialization: Initialize the array with a specified initial capacity.
 - Insertion: Add elements to the array, expanding capacity as necessary.
 - Deletion: Remove elements from the array and compact the memory.
 - Access: Access elements by index.
 - Resize: Dynamically resize the array to optimize memory usage.

2. Memory Management:

- Implement a custom memory allocator within your dynamic array to handle memory allocation and deallocation.
- Optimize memory reallocation strategy to minimize memory fragmentation and overhead.
- Implement mechanisms to avoid memory leaks, dangling pointers, and ensure memory alignment suitable for the target architecture.

3. Advanced Memory Handling:

- Implement a garbage collection mechanism to reclaim memory from deleted elements.
- Use memory pools to manage small memory allocations efficiently.

4. Concurrency:

- Implement thread-safe versions of insertion, deletion, and access operations.
- Use synchronization primitives (e.g., mutexes) to ensure thread safety.
- Implement a lock-free version of the container for high-performance scenarios.

5. Error Handling:

- Include comprehensive error handling for out-of-memory situations, invalid index access, and other potential errors.
- Develop a custom error handling framework that logs errors to a file with timestamps, ensuring that error messages are descriptive and include context about the error.

6. Edge Cases:

- Ensure the implementation handles edge cases such as adding elements beyond initial capacity, deleting from an empty array, accessing elements out of bounds, and resizing under heavy load.

7. Performance Considerations:

- Optimize the implementation for both time and space complexity.
- Write a benchmarking program to compare the performance of your custom container against `std::vector` in C++.
- Provide a report with the benchmarking results and analysis.
- Implement a mechanism to profile and optimize memory access patterns.

8. Security Considerations:

- Implement checks to prevent buffer overflow and other common vulnerabilities.
- Ensure safe handling of untrusted inputs.

Constraints:

- The entire solution should be implemented in pure C (C89/C90 standard).
- No use of external libraries except for standard C library functions.
- The code should be well-documented, with comments explaining key sections and decisions.

Deliverables:

1. **Source Code:** A `.c` file containing the implementation of the custom dynamic array.
2. **Header File:** A `.h` file with the declarations of the functions and data structures used.
3. **Test Cases:** A separate `.c` file with a set of comprehensive test cases demonstrating the functionality and robustness of your implementation.
4. **Benchmarking Program:** A `.c` file with the benchmarking code and a report on performance comparison.
5. **Readme:** A `README.md` file explaining how to compile and run the code, along with an overview of the design and optimization strategies.

Example Interface:

```
typedef struct {
    int *data;
    size_t size;
    size_t capacity;
    pthread_mutex_t lock;
} CustomArray;

void initArray(CustomArray *array, size_t initial_capacity);
void insertElement(CustomArray *array, int element);
void deleteElement(CustomArray *array, size_t index);

int getElement(CustomArray *array, size_t index);
void freeArray(CustomArray *array);
```

Grading Criteria:

- **Correctness:** The implementation correctly performs all required operations.
-

Memory Management: Efficient use of memory and proper handling of memory allocation/deallocation.

- **Robustness:** Comprehensive error handling and handling of edge cases.
- **Performance:** Optimized for both time and space complexity.
- **Concurrency:** Effective implementation of thread safety and lock-free operations.
- **Security:** Implementation of security measures to prevent common vulnerabilities.
- **Code Quality:** Clear, maintainable code with proper documentation and comments.
- **Advanced Features:** Implementation of advanced requirements and quality of the chosen strategies.